Nama : Elsadanika Maura Sitorus
NIM : 211344013 – 2NK

# RANGKUMAN MINGGU 2 MIKROPROSESOR & INTERFACE

Assembler define digunakan sebagai alokasi ruang penyimpanan dan untuk menginisialisasi satu atau lebih byte. Nama variabel digunakan untuk ruang penyimpanan yang terkait dengan nilai offset dalam data segmen. Berikut adalah bentuk-bentuk dasar dari directive define:

| Perintah | Keterangan | Storage Space |
|---|---|---|
| DB | Define Byte | 1 byte |
| DW | Define Word | 2 bytes |
| DD | Define Doubleword | 4 bytes |
| DQ | Define Quadword | 8 bytes |
| DT | Define Ten Bytes | 10 bytes |

Instruksi untuk transfer perintah selama eksekusi disesuaikan dengan kondisinya, berikut ini adalah instruksi untuk proses looping dalam Bahasa assembly pada 8086:

| Instructions | Conditions |
|---|---|
| JA/JNBE | Jump if above/not below/equal instruction satisfies |
| JAE/JNB | Jump if above/not below |
| JBE/JNA | Jump if below/equal/ not above |
| JC | Jump if carry flag CF = 1 |
| JE/JZ | Jump if equal/zero flag ZF = 1 |
| JG/JNLE | Jump if greater/not less than/equal |
| JGE/JNL | Jump if greater than/equal/not less |
| JL/JNGE | Jump if less than/not greater than/equal |
| JLE/JNG | Jump if less than/equal/if not greater |
| JNC | Jump if no carry flag (CF = 0) |
| JNE/JNZ | Jump if not equal/zero flag ZF = 0 |
| JNO | Jump if no overflow flag OF = 0 |
| JNP/JPO | Jump if not parity/parity odd PF = 0 |
| JNS | Jump if not sign SF = 0 |
| JO | Jump if overflow flag OF = 1 |
| JP/JPE | Jump if parity/parity even PF = 1 |
| JS | Jump if sign flag SF = 1 |

Instruksi-instruksi pendukung untuk membuat loop pada program menjumlahkan heksadesimal pada EMU 8086 Simulator.

| **Instruction** | CMP | SUB | Label |
|---|---|---|---|

| **Condition** | (Compare) Hasil pengurangan tidak disimpan dimanapun. | (Substraction) Hasil pengurangan disimpan di akumulator. | Memberi label pada program. |
|---|---|---|---|
| **Example** | CMP AL, CL | SUB AL, CL | LUP: |

Jika terdapat data: FFFF, DD7A. Untuk data 16 bit maka SI akan bernilai 2. Maka mekanisme pada penyimpanan atau storage adalah sebagai berikut.

| Initialization | Source Index | Data | Storage Space | Data Input |
|---|---|---|---|---|
| BIL | 0 | FF | 8 bit | FFFF |
| | 1 | FF | 8 bit | |
| | 2 | 7A | 8 bit | DD7A |
| | 3 | DD | 8 bit | |

**Penjumlahan Decimal**

Penjumlahan decimal harus diberi faktor koreksi, yaitu penambahan 0110 (enam). Berikut ini adalah contoh penjumlahan decimal dari 78+94 = 172.

| | | |
|---|---|---|
| | 0111 | 1000 |
| | 1001 | 0100 |
| ---------- + | | |
| 0001 | 0000 | 1100 |
| 0000 | 0110 | 0110 |
| ---------- + | | |
| 0001 | 0111 | 0010 |

Faktor koreksi diberikan ketika hasil penjumlahan biner bernilai > 9 dan/atau memberikan carry ke penjumlahan selanjutnya. Pada intel 8086, pemberian faktor koreksi menggunakan instruksi DAA. DAA memberi faktor koreksi supaya hasil penjumlahan yang dilakukan menjadi benar secara decimal.

Hanya register AL yang dapat digunakan untuk operasi dengan melibatkan DAA. Selain itu, pada angka yang dijumlahkan harus menggunakan “H” diakhir angkanya atau “0X” di awal angka seperti contoh 98H atau 0X98, karena jika tidak maka akan dianggap decimal dan dikonversikan ke biner. Data decimal wajib menggunakan instruksi DAA, dan DAA hanya berlaku pada register AL.

Untuk menjumlahkan data 16 bit, akan lebih mudah dipahami mekanisme penjumlahannya dengan menggunakan tabel memory penyimpanan. Sebagai contoh, jika ingin menjumlahkan 3 data 16 bit bernilai 3399,7799,1234 maka penjumlahannya akan sebagai berikut.

| | | |
|---|---|---|
| 0 | 99 | |
| 1 | 33 | (99+99) → [DI + BIL] |
| 2 | 99 | (33+77) → [BIL+DI+1] |
| 3 | 77 | |
| 4 | 34 | → low byte |
| 5 | 12 | → high byte |

Jika dimiliki data yang bernilai sama (ex: 9999) sebanyak 20 data disimpan pada register BL, CH, CL. Maka dapat dibuat program untuk menjumlahkannya dengan menggunakan loop. Berikut ini adalah contoh program dan output yang dihasilkan.

```
01 TITLE PENJ 20 DATA DES 4 DIGIT
02 .MODEL SMALL
03 .STACK 100H
04 .DATA
05 BIL DB 40 DUP(99h)
06 .CODE
07 MAIN PROC
08      MOV CX,@DATA
09      MOV DS,CX
10      MOV BL,0
11      MOV CX,0
12      MOV DI,0
13 LUP:MOV AL,CL            ;BUAT PENJUM DI CL
14      ADD AL,[BIL+DI]
15      DAA
16      MOV CL,AL           ;DISIMPAN DI CL
17      INC DI
18      MOV AL,CH           ;BUAT PENJUM DI CH
19      ADC AL,[BIL+DI]
20      DAA
21      MOV CH,AL           ;DISIMPAN DI CH
22      MOV AL,BL           ;BUAT PENJUM DI BL
23      ADC AL,0
24      DAA
25      MOV BL,AL           ;DISIMPAN DI BL
26      INC DI
27      CMP DI,40           ;CEK DI
28      JNZ LUP
29
30      MOV AH,4CH          ;STOP
31      INT 21H
32      MAIN ENDP
33 END MAIN
```

